Chapitre 2 - Intégration continue

1. Maven

Qu'est-ce que Maven ?

- Maven est un **outil open-source de gestion et de construction de projets**, développé par la fondation Apache.
- Il facilite :
  - La **gestion des dépendances**
  - La **compilation**
  - Le **packaging**
  - Le **déploiement** des applications
- Maven utilise un **Project Object Model (POM)**, décrit dans un fichier XML (`pom.xml`), qui définit :
  - La structure du projet
  - Les dépendances
  - La configuration du build

Fonctionnalités principales

1. **Cycle de vie automatisé** : Maven définit des phases standard telles que :
   a. `compile` → compiler le code
   b. `test` → exécuter les tests unitaires
   c. `package` → générer le package (JAR, WAR, etc.)
2. **Gestion des dépendances** : Télécharge automatiquement les bibliothèques nécessaires depuis des référentiels centralisés.
3. **Standardisation** : Imposer une structure de projet cohérente pour tous les projets.
4. **Configuration via POM** :
   a. `pom.xml` contient les métadonnées, dépendances et configuration du build.

2. Jenkins

Présentation

- Jenkins est un **serveur d'automatisation open-source écrit en Java**.
- Il est utilisé pour :
  - **Intégration Continue (CI)** : intégrer automatiquement les modifications de code dans un dépôt central, compiler et tester.
  - **Livraison Continue (CD)** : déployer automatiquement les applications dans différents environnements (tests, préproduction, production).

Pipeline Jenkins

- Un **pipeline** est un ensemble de plugins et de configurations qui définissent la séquence automatisée de **build, test et déploiement**.

TP1+TP2

vagrant init # crée le fichier Vagrantfile

vagrant up # télécharge (si besoin) et démarre la VM

vagrant ssh # se connecter à la VM Ubuntu

exit # quitter la VM

vagrant halt # arrêter la VM

vagrant up # démarrer la VM

vagrant ssh # entrer dans la VM

sudo apt update # mise à jour

sudo apt install openjdk-21-jdk –y # installer JDK

sudo apt install maven -y # installer Maven

sudo apt install git -y # installer Git

curl -fsSL https://pkg.jenkins.io/debian/jenkins.io-2023.key | sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null # clé Jenkins

echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/ | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null # repo Jenkins

sudo apt-get install jenkins # installer Jenkins

sudo systemctl start jenkins.service # démarrer Jenkins

sudo systemctl status jenkins # statut Jenkins

sudo cat /var/lib/jenkins/secrets/initialAdminPassword # mot de passe Jenkins

sudo systemctl restart jenkins.service # redémarrer Jenkins

Plugins Jenkins
• **Plugins de build** : Maven Integration, Pipeline Integration
• **Plugins d'analyse de code** : Sonargraph, SonarQube Scanner
• **Plugins de gestion de sources** : Git Plugin
• **Plugins pour configurations, métriques et conteneurs** : Prometheus, Docker Pipeline
• Jenkins dispose de plus de 1500 plugins pour gérer diverses tâches.
 	
3. Agents Jenkins
Architecture
• Jenkins utilise une **architecture maître-esclave (master-agent)** :
  o **Master (maître)** : planifie et orchestre les tâches
  o **Agents (esclaves)** : exécutent les tâches sur différentes machines
• Objectifs :
  o Répartir la charge de travail (plusieurs builds simultanés)
  o Exécuter les builds sur différents environnements (Linux, Windows, conteneurs, etc.)	

Chapitre 3 – Livraison continue

1. Évolution de la machine de production

- **Ancienne méthode** : Une seule application par serveur physique.
- **Limites de cette approche** :
  - Gaspillage de ressources matérielles
  - Manque de flexibilité
  - Gestion complexe
  - Coûts élevés

Architecture traditionnelle : un serveur physique = une seule application

2. Virtualisation

Définition

- La virtualisation permet de **simuler plusieurs machines** sur un seul serveur physique.
- Chaque **machine virtuelle (VM)** a son **propre système d'exploitation** et exécute les applications à l'intérieur.

Virtualisation lourde

- Technique permettant de faire fonctionner **plusieurs OS sur un seul serveur physique**.
- Chaque VM est isolée grâce à un **hyperviseur**.
- Objectif : optimiser l'utilisation des ressources et réduire les coûts en consolidant plusieurs serveurs physiques en un seul.

Avantages :

- Meilleure exploitation des ressources physiques
- Plusieurs machines virtuelles peuvent partager un serveur

Limites :

- Chaque VM consomme CPU, RAM, stockage et a besoin de son propre OS invité
- Plus il y a de VMs, plus la consommation de ressources augmente

Virtualisation légère ou conteneurisation

- Basée sur des **conteneurs isolés** sur un **OS commun**.
- Les applications partagent le même système d'exploitation mais restent isolées les unes des autres.
- Les conteneurs permettent de **coexister sur un même OS et partager la mémoire**, ce qui économise des ressources.
- Un **conteneur** est une "enveloppe" qui contient tout ce dont une application a besoin pour fonctionner : code, bibliothèques, outils système, configuration.

3. Docker

Définition

- Docker est une **plateforme open-source** qui permet de créer, déployer et exécuter des applications dans des **conteneurs logiciels**.

TP3

```
sudo apt install apt-transport-https ca-certificates curl software-properties-common -y # deps Docker
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add - # clé Docker
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu focal stable" # repo Docker
sudo apt install docker-ce -y   # installer Docker
sudo systemctl status docker   # vérifier Docker (CTRL+C pour quitter)

sudo chmod 666 /var/run/docker.sock     # donner droits d'accès
docker -v                # version Docker
docker run hello-world        # test Docker

docker pull image:version      # télécharger une image
docker run image            # créer un conteneur
docker images              # voir images locales
docker ps -a               # voir conteneurs
docker run -itd user/alpine:1.0.0 /bin/sh # lancer conteneur en background
docker exec -it conteneurID /bin/sh     # accéder au conteneur
docker image rm image --force   # supprimer une image
docker rm conteneurID        # supprimer un conteneur

docker login               # connexion DockerHub
docker push user/image:tag     # envoyer image vers DockerHub
```

• Les conteneurs sont **standardisés**, encapsulent tout ce dont une application a besoin, et permettent une **exécution cohérente** sur différents environnements (développement, test, production). **Lexique Docker** • **Conteneur** : instance d'une image, exécutable, contenant l'environnement complet nécessaire à l'application. • **Image** : modèle read-only de conteneur, incluant OS de base, application et dépendances. • **Docker Hub** : dépôt public d'images Docker accessibles à tous o Lien : Docker Hub **Résumé :** un conteneur = instance d'une image Docker exécutée avec toutes ses dépendances.	

Chapitre 4 – Vérification de la qualité du code

1. Introduction

Objectif

- S'assurer que le code des logiciels est de **haute qualité dès le début du développement**.
- Garantir que les applications restent **fiables et sécurisées à long terme**.
- Environ **30% du temps de développement** doit être consacré aux tests pour vérifier :
  - La qualité du code
  - La sécurité
  - L'absence de bugs

Types de tests : dynamiques et statiques

Tests dynamiques

- Réalisés **lorsque l'application tourne**.
- Objectif : détecter les **dysfonctionnements** tels que :
  - Fonctionnalités mal implémentées
  - Base de données inaccessible
- Exemples :
  - Tests unitaires
  - Tests d'intégration
  - Tests de régression

Tests statiques

- Réalisés **sur le code source avant son exécution**.
- Objectif : détecter les **écarts aux bonnes pratiques** de développement, comme :
  - Absence de logs
  - Absence de commentaires
- **SonarQube** est un outil spécialisé dans ce type de tests.

2. SonarQube

Présentation

- Outil **open-source** d'**analyse statique du code**.
- Permet de **vérifier en continu la qualité du code source** selon des règles prédéfinies.
- Fonctionne comme une **revue de code automatique**, aidant les équipes à maintenir un code propre, sécurisé et fiable.

Avantages

- Détection précoce des **erreurs et vulnérabilités**
- Suivi de la **qualité du code dans le temps**
- Amélioration de la **maintenabilité et lisibilité du code**
- Support de plusieurs **langages de programmation**

TP4

sudo chmod 666 /var/run/docker.sock
docker pull sonarqube
docker images docker run -d -p 9000:9000 sonarqube
docker ps –a
docker start<id_conteneur_sonarqube>

stage('MAVEN Build') { steps { // Lance Maven pour nettoyer et compiler le projet // 'clean' supprime les anciens fichiers // 'compile' compile le code source sh 'mvn clean compile' } }

stage('SONARQUBE') { environment { // URL de ton serveur SonarQube
SONAR_HOST_URL = 'http://192.168.50.4:9000/'

```
   // Récupération du token stocké dans Jenkins Credentials
    SONAR_AUTH_TOKEN = credentials('sonarqube')
}

steps {
    // Analyse du projet avec SonarQube
    // -Dsonar.projectKey : identifiant du projet dans Sonar
    // -Dsonar.host.url : lien vers le serveur SonarQube
    // -Dsonar.token : authentification avec le token
    sh "mvn sonar:sonar -Dsonar.projectKey=devops_git -
Dsonar.host.url=$SONAR_HOST_URL -Dsonar.token=$SONAR_AUTH_TOKEN"
}


}
```

Chapitre 5 – Supervision Continue

1. Introduction

La **supervision continue** (ou monitoring continu) est essentielle dans tout environnement DevOps.
Elle permet de garantir la **stabilité**, la **performance** et la **fiabilité** des applications et infrastructures.

Objectifs principaux :

✓ **Détection** rapide **des problèmes**

✓ **Réaction rapide aux incidents**

✓ **Automatisation des réparations**

- Possibilité de déclencher automatiquement des scripts de remediation :
  - redémarrage d'un service
  - redéploiement d'un conteneur

✓ **Amélioration continue**

✓ **Réduction du temps d'arrêt (downtime)** les interruptions de service

2. Outils de supervision

De nombreux outils existent selon les besoins : infrastructure, logs, performances, alertes…

✓ **Prometheus**

- Outil de monitoring open-source orienté métriques.
- Collecte de données depuis des services, conteneurs, applications, etc.
- Fonctionne avec un système de "pull" (scraping).

✓ **Grafana**

- Plateforme de visualisation des métriques.
- Crée des dashboards personnalisés et dynamiques.

TP5

```
sudo chmod 666 /var/run/docker.sock
docker run -d --name prometheus -p 9090:9090 prom/prometheus
docker exec -it prometheus sh
docker restart prometheus
http://<IP_VM>:9090/targets
http://<IP_VM>:8080/prometheus/

docker run -d --name grafana -p 3000:3000 grafana/grafana
http://<IP_VM>:3000 # Accéder à Grafana
```

Prometheus.yml.txt :

```
tee -a /etc/prometheus/prometheus.yml <<EOF
# Nom du job dans Prometheus
- job_name: FirstPipeline

 # Chemin où Jenkins expose ses métriques (grâce au plugin Prometheus)
 metrics_path: /prometheus

 # Liste des cibles à surveiller
 static_configs:
  # L'IP et le port de Jenkins (8080)
 - targets: ['192.168.50.4:8080']
EOF
```

Chapitre 6 – Orchestration des conteneurs

Introduction

- Kubernetes : plateforme open source d'orchestration des conteneurs.
- Automatise la configuration, le déploiement et la gestion des applications.
- Regroupe code + configuration + dépendances dans des environnements isolés.

Fonctionnalités principales

- **Orchestration** : gestion de milliers de conteneurs.
- **Scalabilité automatique** : adaptation selon la charge.
- **Haute disponibilité** : redémarrage automatique en cas de panne.
- **Rolling updates** : mises à jour sans interruption.
- **Gestion réseau / stockage** : communication entre conteneurs et stockage persistant.

Architecture Kubernetes

- Un **cluster** = plan de contrôle (Master) + nœuds de calcul (Workers).
- Le **plan de contrôle** maintient l'état souhaité du cluster et planifie les pods.
- Les **nœuds** fournissent les ressources CPU / RAM pour exécuter les pods.

Concepts de base

Pods

- Plus petite unité déployable.
- Contient un ou plusieurs conteneurs.
- Partage réseau + stockage.
- Les pods du même nœud ont la même IP.

Service

- Donne une **IP fixe + DNS** pour accéder aux pods.
- Assure le **load balancing** entre pods.

Deployment

- Déploie et maintient automatiquement les pods.
- Haute disponibilité.
- Auto-réparation.
- Rolling updates + rollback.
- Scaling horizontal facile.

TP6

deployment.yaml :

1. Créer le fichier « deployment.yaml »

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: test-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: test-app
  template:
    metadata:
      labels:
        app: test-app
    spec:
      containers:
        - name: test-app
          image: tarekayari1/test-app:1.0.0
          ports:
            - containerPort: 80
```

service.yaml :

1. Créer le fichier « service.yaaml »

```yaml
apiVersion: v1
kind: Service
metadata:
  name: test-service
spec:
  type: NodePort
  selector:
    app: test-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30036
```

TP6

curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64 # Télécharge Minikube

sudo install minikube-linux-amd64 /usr/local/bin/minikube # Installe Minikube

rm minikube-linux-amd64 # Supprime le fichier

```
curl -LO https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl  # Télécharge kubectl
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl                # Installe kubectl
rm kubectl                                                                          # Supprime le fichier

minikube start --driver=docker      # Démarre Minikube
minikube version                    # Affiche la version Minikube
minikube status                     # Vérifie l'état du cluster
kubectl version --client            # Affiche la version de kubectl
kubectl config get-contexts         # Montre les contextes Kubernetes configurés

docker run -it --rm -d -p 5000:80 --name web nginx          # Lance un conteneur Nginx exposé sur le port 5000
docker stop web                                             # Stoppe le conteneur Nginx
mkdir Kubernetes                                            # Crée le répertoire de travail
# créer index.html dans Kubernetes/                         # Éditer manuellement index.html
docker run -it --rm -d -p 5000:80 --name web -v ~/kubernet:/usr/share/nginx/html nginx  # Monte le dossier local

docker build . -t test-app          # Construit l’image Docker "test-app"
docker run -p 8085:80 -d test-app   # Teste l’image en lançant le conteneur sur port 8085

docker login -u tarekayari1         # Connexion à Docker Hub
docker tag test-app:latest tarekayari1/test-app:1.0.0   # Renomme l’image avec un tag
docker push tarekayari1/test-app:1.0.0                  # Envoie l’image sur Docker Hub
```

```
kubectl apply -f deployment.yaml      # Déploie l’image dans Kubernetes

kubectl get pods                      # Liste les pods pour vérifier leur statut

kubectl get deployments               # Liste les deployments


kubectl apply -f service.yaml         # Crée l'objet Service

kubectl get services                  # Liste les services

minikube service hello-service --url    # Génère une URL externe pour accéder à l'application
```